

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I M. Varshith Reddy(192524174) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Varshith Reddy

Annexure A

Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets (≥ 10 MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange

A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.

- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation
--------------	-----	--	--	---	---

```
In [ ]: pip install pycryptodome
```

Collecting pycryptodome

Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)

Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)

```
0.0/2.3 MB ? eta -:-:--
2.3/2.3 MB 83.9 MB/s eta 0:00:01
2.3/2.3 MB 46.6 MB/s eta 0:00:00
```

Installing collected packages: pycryptodome

Successfully installed pycryptodome-3.23.0

```
In [ ]: from Crypto.Cipher import DES, DES3
from Crypto.Util.Padding import pad, unpad
import time

# -----
# DES Encryption
# -----
def des_encrypt(key, plaintext):
    cipher = DES.new(key.encode(), DES.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext.encode(), DES.block_size))
    return ciphertext

def des_decrypt(key, ciphertext):
    cipher = DES.new(key.encode(), DES.MODE_ECB)
    plaintext = unpad(cipher.decrypt(ciphertext), DES.block_size)
    return plaintext.decode()

# -----
# Triple DES Encryption
# -----
def triple_des_encrypt(key, plaintext):
    key = DES3.adjust_key_parity(key.encode())
    cipher = DES3.new(key, DES3.MODE_ECB)
    ciphertext = cipher.encrypt(pad(plaintext.encode(), DES3.block_size))
    return ciphertext

def triple_des_decrypt(key, ciphertext):
    key = DES3.adjust_key_parity(key.encode())
    cipher = DES3.new(key, DES3.MODE_ECB)
    plaintext = unpad(cipher.decrypt(ciphertext), DES3.block_size)
    return plaintext.decode()

# -----
# Main Program
# -----
print("==== DES Encryption ====")
des_key = input("Enter 8-character DES key: ")
plaintext = input("Enter plaintext: ")

start = time.time()
des_cipher = des_encrypt(des_key, plaintext)
des_time = time.time() - start

print("\nDES Ciphertext (Hex):", des_cipher.hex())

start = time.time()
des_plain = des_decrypt(des_key, des_cipher)
des_dec_time = time.time() - start

print("DES Decrypted Text:", des_plain)

print("\nDES Encryption Time:", des_time, "seconds")
print("DES Decryption Time:", des_dec_time, "seconds")

print("\n=====")

print("\n==== Triple DES Encryption ====")
tdes_key = input("Enter 24-character 3DES key: ")

start = time.time()
tdes_cipher = triple_des_encrypt(tdes_key, plaintext)
tdes_time = time.time() - start

print("\n3DES Ciphertext (Hex):", tdes_cipher.hex())

start = time.time()
tdes_plain = triple_des_decrypt(tdes_key, tdes_cipher)
tdes_dec_time = time.time() - start

print("3DES Decrypted Text:", tdes_plain)
```

```
print("\n3DES Encryption Time:", tdes_time, "seconds")
print("3DES Decryption Time:", tdes_dec_time, "seconds")
```

===== DES Encryption =====

Enter 8-character DES key: 12345678

Enter plaintext: HelloWorld

DES Ciphertext (Hex): d3b0f050b34be8225d82e3681c83bb77

DES Decrypted Text: HelloWorld

DES Encryption Time: 0.004063129425048828 seconds

DES Decryption Time: 0.0005807876586914062 seconds

=====

===== Triple DES Encryption =====

Enter 24-character 3DES key: 123456789012345678901234

3DES Ciphertext (Hex): c09be09533656a2f3825219b9e150c2e

3DES Decrypted Text: HelloWorld

3DES Encryption Time: 0.0003838539123535156 seconds

3DES Decryption Time: 0.0005934238433837891 seconds

```
In [4]: from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes
import time

# -----
# User Input
# -----
key = input("Enter 16-character AES Key: ").encode()

while len(key) != 16:
    print("Key must be exactly 16 characters.")
    key = input("Enter 16-character AES Key: ").encode()

plaintext = input("Enter Plaintext: ").encode()

iv = get_random_bytes(16)

# -----
# ECB MODE
# -----
print("\n===== ECB MODE =====")

cipher = AES.new(key, AES.MODE_ECB)

start = time.time()
ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
end = time.time()

print("Ciphertext:", ciphertext.hex())

cipher = AES.new(key, AES.MODE_ECB)
decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)

print("Decrypted:", decrypted.decode())
print("Encryption Time:", end-start)

# -----
# CBC MODE
# -----
print("\n===== CBC MODE =====")

cipher = AES.new(key, AES.MODE_CBC, iv)

start = time.time()
ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
end = time.time()

print("Ciphertext:", ciphertext.hex())

cipher = AES.new(key, AES.MODE_CBC, iv)
decrypted = unpad(cipher.decrypt(ciphertext), AES.block_size)

print("Decrypted:", decrypted.decode())
print("Encryption Time:", end-start)

# -----
# CFB MODE
# -----
print("\n===== CFB MODE =====")
```

```

cipher = AES.new(key, AES.MODE_CFB, iv)

start = time.time()
ciphertext = cipher.encrypt(plaintext)
end = time.time()

print("Ciphertext:", ciphertext.hex())

cipher = AES.new(key, AES.MODE_CFB, iv)
decrypted = cipher.decrypt(ciphertext)

print("Decrypted:", decrypted.decode())
print("Encryption Time:", end-start)

# -----
# OFB MODE
# -----
print("\n===== OFB MODE =====")

cipher = AES.new(key, AES.MODE_OFB, iv)

start = time.time()
ciphertext = cipher.encrypt(plaintext)
end = time.time()

print("Ciphertext:", ciphertext.hex())

cipher = AES.new(key, AES.MODE_OFB, iv)
decrypted = cipher.decrypt(ciphertext)

print("Decrypted:", decrypted.decode())
print("Encryption Time:", end-start)

```

Enter 16-character AES Key: ABCD1234EFGH5678

Enter Plaintext: ComputerNetworks

===== ECB MODE =====

Ciphertext: 8f6edcb8d820f1aca7ee5c8d977693dcdf3f2ff0eb9a4fb69d2a2272e469015d

Decrypted: ComputerNetworks

Encryption Time: 0.00016570091247558594

===== CBC MODE =====

Ciphertext: 9b7fbb2bb8cc63e7af745f16a16e1baa589e06a529bf4cda9e0139ee65c4c7d5

Decrypted: ComputerNetworks

Encryption Time: 0.00023794174194335938

===== CFB MODE =====

Ciphertext: 9104f9382fe670f7f1e246895d18c05e

Decrypted: ComputerNetworks

Encryption Time: 0.0003273487091064453

===== OFB MODE =====

Ciphertext: 91d4b76a2620f9994213f406237a791c

Decrypted: ComputerNetworks

Encryption Time: 0.00015163421630859375

```

In [5]: from sympy import randprime
        from math import gcd

# -----
# Generate Prime Numbers
# -----
p = randprime(100, 500)
q = randprime(100, 500)

while p == q:
    q = randprime(100, 500)

print("Generated Prime Numbers")
print("P =", p)
print("Q =", q)

# -----
# Calculate n and phi
# -----
n = p * q
phi = (p - 1) * (q - 1)

# -----
# Choose Public Key
# -----
e = 2
while gcd(e, phi) != 1:

```

```

    e += 1

# -----
# Find Private Key
# -----
d = pow(e, -1, phi)

print("\nPublic Key (e, n):")
print("( ", e, ", ", n, " )")

print("\nPrivate Key (d, n):")
print("( ", d, ", ", n, " )")

# -----
# User Input
# -----
message = int(input("\nEnter Message (Number less than n): "))

while message >= n:
    print("Message must be smaller than", n)
    message = int(input("Enter Message Again: "))

# -----
# Encryption
# -----
cipher = pow(message, e, n)

print("\nEncrypted Message =", cipher)

# -----
# Decryption
# -----
plain = pow(cipher, d, n)

print("Decrypted Message =", plain)

```

Generated Prime Numbers

P = 211

Q = 487

Public Key (e, n):

(11 , 102757)

Private Key (d, n):

(46391 , 102757)

Enter Message (Number less than n): 12345

Encrypted Message = 85773

Decrypted Message = 12345

In [6]: # Diffie-Hellman Key Exchange

```

# Public values
p = int(input("Enter a Prime Number (P): "))
g = int(input("Enter Primitive Root (G): "))

# Alice's Private Key
a = int(input("Enter Alice's Private Key: "))

# Bob's Private Key
b = int(input("Enter Bob's Private Key: "))

# Alice's Public Key
A = pow(g, a, p)

# Bob's Public Key
B = pow(g, b, p)

print("\nAlice's Public Key =", A)
print("Bob's Public Key =", B)

# Shared Secret
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)

print("\nAlice's Shared Secret =", alice_secret)
print("Bob's Shared Secret =", bob_secret)

if alice_secret == bob_secret:
    print("\nKey Exchange Successful")
else:
    print("\nKey Exchange Failed")

```


Enter a Prime Number (P): 23
Enter Primitive Root (G): 5
Enter Alice's Private Key: 6
Enter Bob's Private Key: 15

Alice's Public Key = 8
Bob's Public Key = 19

Alice's Shared Secret = 2
Bob's Shared Secret = 2

Key Exchange Successful

In [8]: `pip install tinyec`

```
Collecting tinyec
  Downloading tinyec-0.4.0.tar.gz (24 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: tinyec
  Building wheel for tinyec (setup.py) ... done
  Created wheel for tinyec: filename=tinyec-0.4.0-py3-none-any.whl size=20878 sha256=b50077f1547f691bda396fdcf6c95de5b5b63fd2303a3badf00fc7a64c2b455d
  Stored in directory: /root/.cache/pip/wheels/87/5b/7c/58c1f0abdaf45dbbe75e09e682d54058fb6bf6fa1458f8897e
Successfully built tinyec
Installing collected packages: tinyec
Successfully installed tinyec-0.4.0
```

```
In [9]: from tinyec import registry
import secrets

# Select Curve
curve = registry.get_curve("brainpoolP256r1")

print("Elliptic Curve:", curve.name)

# -----
# Alice's Private Key
# -----
alice_private = int(input("Enter Alice's Private Key: "))

# -----
# Bob's Private Key
# -----
bob_private = int(input("Enter Bob's Private Key: "))

# -----
# Public Keys
# -----
alice_public = alice_private * curve.g
bob_public = bob_private * curve.g

print("\nAlice Public Key:")
print(alice_public)

print("\nBob Public Key:")
print(bob_public)

# -----
# Shared Secret
# -----
alice_shared = alice_private * bob_public
bob_shared = bob_private * alice_public

print("\nAlice Shared Key:")
print(alice_shared)

print("\nBob Shared Key:")
print(bob_shared)

# -----
# Verification
# -----
if alice_shared == bob_shared:
    print("\nECC Key Exchange Successful")
else:
    print("\nECC Key Exchange Failed")
```

Elliptic Curve: brainpoolP256r1
Enter Alice's Private Key: 15
Enter Bob's Private Key: 22

Alice Public Key:

(1894829957716799558394405977062072639705660958709100150317503973453175552654, 19325587742235095167029324831837419109162517315763469724500492428133372801981) on "brainpoolP256r1" $\Rightarrow y^2 = x^3 + 56698187605326110043627228396178346077120614539475214109386828188763884139993x + 17577232497321838841075697789794520262950426058923084567046852300633325438902 \pmod{76884956397045344220809746629001649093037950200943055203735601445031516197751}$

Bob Public Key:

(63057025950715569420853016967530468131237476850801479529493393913530481531883, 16891068814153325345512844365427615461513351423782589856118738953089921010047) on "brainpoolP256r1" $\Rightarrow y^2 = x^3 + 56698187605326110043627228396178346077120614539475214109386828188763884139993x + 17577232497321838841075697789794520262950426058923084567046852300633325438902 \pmod{76884956397045344220809746629001649093037950200943055203735601445031516197751}$

Alice Shared Key:

(58586432099128228312854603752476818817545834282720336459235856366825631890631, 66966255260153346527727176211542439043654365118832116294318711579310707903470) on "brainpoolP256r1" $\Rightarrow y^2 = x^3 + 56698187605326110043627228396178346077120614539475214109386828188763884139993x + 17577232497321838841075697789794520262950426058923084567046852300633325438902 \pmod{76884956397045344220809746629001649093037950200943055203735601445031516197751}$

Bob Shared Key:

(58586432099128228312854603752476818817545834282720336459235856366825631890631, 66966255260153346527727176211542439043654365118832116294318711579310707903470) on "brainpoolP256r1" $\Rightarrow y^2 = x^3 + 56698187605326110043627228396178346077120614539475214109386828188763884139993x + 17577232497321838841075697789794520262950426058923084567046852300633325438902 \pmod{76884956397045344220809746629001649093037950200943055203735601445031516197751}$

ECC Key Exchange Successful